
UNIVERSITY OF MICHIGAN

EECS 471

APPLIED PARALLEL PROGRAMMING WITH GPUS

Final Project

December 7, 2022

Author: Chengfan Li

Unique name: llllcf

Author: Minjie Mao

Unique name: mminjie

1 Introduction

We implemented 4 optimizations to two forward pass convolutional layers of the neural network step by step, achieving 0.092179s for both two layers combined, ranking the third place on the leaderboard. In the first step, we exploit the parallelism of GPU by making each thread compute one element of the output. This optimization leads to a huge improvement of the performance for the naive version. Then the unrolling technique is used to expand the input features into a single 2D matrix, where a highly efficient matrix multiplication is exploited to compute the results of the forward layer. To further improve the performance, two different kernel implementations are carefully designed based on the features of two layers in the neural network. The final improvement is made by sweeping through various parameters, including tile size and block dimensions, to find the best configuration for those kernels.

2 Optimization 1: Exploiting parallelism

2.1 Description

The convolution computation can first be assigned to more blocks and kernels for acceleration. A naive method is introduced in Figure 16.12 in the textbook. The block dimension is (TILE_WIDTH, TILE_WIDTH) and the grid dimension is (B, M, Z) where B is the batch size corresponding to samples in the batch, M is the output feature map size and Z is the number of blocks to be allocated for each image which can be calculated by

$$Z = \text{ceil}(H_{out}/\text{TILE_WIDTH}) \times \text{ceil}(W_{out}/\text{TILE_WIDTH}).$$

So that the third dimension of the grid can be used to find the location of the output tile relative to the image.

Then each thread will use three loops through input channel number, convolution kernel height and convolution kernel width to calculate the corresponding output element. And the mapping relationship between thread and element can be expressed as.

$$\begin{aligned} W_{grid} &= \text{ceil}(W_{out}/\text{TILE_WIDTH}); \\ h &= (\text{blockIdx.z}/W_{grid}) \times \text{TILE_WIDTH} + \text{threadIdx.y}; \\ w &= (\text{blockIdx.z} \% W_{grid}) \times \text{TILE_WIDTH} + \text{threadIdx.x}; \\ y4d(\text{blockIdx.x}, \text{blockIdx.y}, h, w) &= \text{res}; \end{aligned}$$

2.2 Profiler Output

```
New Inference
Op Time: 0.044929
Op Time: 0.115083
Correctness: 0.7955 Model: eeecs498
==3070424== Profiling application: python2 submit/submission.py
==3070424== Profiling result:
      Type  Time(%)   Time     Calls      Avg      Min      Max   Name
GPU activities:  58.76%  159.96ms       2  79.982ms  44.898ms  115.07ms  mxnet::op::forward_kernel(1
                  31.51%  85.787ms       8  10.723ms  1.1200us  83.670ms  [CUDA memcpy HtoD]
                  4.12%  11.215ms       2  5.6074ms  1.9044ms  9.3104ms  void mshadow::cuda::MapPlar
                  2.22%   6.0529ms       1  6.0529ms  6.0529ms  6.0529ms  volta_sgemm_32x32_sliced1x4
                  1.97%   5.3592ms       2  2.6796ms  19.169us  5.3400ms  void op_generic_tensor_kerr
                  1.22%   3.3162ms       1  3.3162ms  3.3162ms  3.3162ms  void cudnn::ops::pooling_fv
                  0.12%   314.62us       1  314.62us  314.62us  314.62us  void mshadow::cuda::MapPlar
                  0.03%   75.585us      13  5.8140us  2.5600us  22.176us  void mshadow::cuda::MapPlar
                  0.02%   56.159us       1  56.159us  56.159us  56.159us  void mshadow::cuda::Softma
```

Figure 1: Optimization 1: Profiler Output

In this optimization, we in total use one kernel function and we can see that most of the time is used for kernel calculation which shows that the main direction for further optimization lies in the algorithm and the reduction of computation per thread. Moreover, when the size of the convolution kernel is small, we can load it into constant memory to save the time for global memory access.

3 Optimization 2: Kernel fusion for unrolling and matrix-multiplication

3.1 Description

We build this faster kernel by reducing the convolution kernel to a **highly efficient matrix multiplication** with the idea of **unfolding and replicating the inputs** to make them compatible for matrix multiplication. We use **shared memory** to implement **tilted matrix multiplication** introduced in the lecture. Using shared memory saves a large amount of time for repeated global memory access. Although the idea is to unroll the feature maps, we do not unroll it explicitly since it requires too much memory if the batch size is large. Given the index (ty, tx) of the element in the shared memory, which corresponds to its index in the unrolled matrix, the kernel computes the actual index in the original input feature maps and directly loads it into the shared memory. The loading is obviously not optimized because the memory coalescing is impossible in this case. However, it saves memory space since the kernel does not actually replicate the inputs. The index from shared memory subTileB is converted to the index of the input x in the following way:

$$\begin{aligned}\text{Col} &= bx * \text{TILE_WIDTH} + tx; \\ \text{tile_x} &= i * \text{TILE_WIDTH} + tx; \\ \text{tile_y} &= i * \text{TILE_WIDTH} + ty; \\ W_{\text{out}} &= W - K + 1; \\ k_{\text{idx}} &= \text{tile_x} \% (K * K); \\ x_{\text{idx}} &= \text{tile_y} \% (K * K); \end{aligned}$$

```

subTileB[ty][tx]
= x4d(bz, tile_y/(K*K), Col/W_out + x_idx/K, Col % W_out + x_idx % K).

```

In the calculation, tx, ty represents the `threadIdx.x` and `threadIdx.y`. The block index in z dimension is bz and i denotes the phase of tiling. The block dimension is (TILE_WIDTH, TILE_WIDTH)

3.2 Profiler Output

```

New Inference
Op Time: 0.058904
Op Time: 0.095432
Correctness: 0.7955 Model: eecs498
==2048420== Profiling application: python2 submit/submission.py
==2048420== Profiling result:

```

Type	Time(%)	Time	Calls	Avg	Min	Max	Name
GPU activities:	36.50%	103.80ms	8	12.976ms	1.1520us	101.68ms	[CUDA memcpy HtoD]
	33.55%	95.413ms	1	95.413ms	95.413ms	95.413ms	mxnet::op::forward_kernel2(float*, float
	20.70%	58.878ms	1	58.878ms	58.878ms	58.878ms	mxnet::op::forward_kernel1(float*, float
	3.94%	11.220ms	2	5.6101ms	1.8925ms	9.3278ms	void mshadow::cuda::MapPlanLargeKernel<m
	2.09%	5.9325ms	1	5.9325ms	5.9325ms	5.9325ms	volta_sgemm_32x32_sliced1x4_tn
	1.87%	5.3329ms	2	2.6664ms	18.560us	5.3143ms	void op_generic_tensor_kernel<int=1, flo
	1.17%	3.3176ms	1	3.3176ms	3.3176ms	3.3176ms	void cudnn::ops::pooling_fw_4d_kernel<fl
	0.11%	311.04us	1	311.04us	311.04us	311.04us	void mshadow::cuda::MapPlanLargeKernel<m
	0.03%	76.897us	13	5.9150us	2.6240us	22.495us	void mshadow::cuda::MapPlanKernel<mshado
	0.02%	55.328us	1	55.328us	55.328us	55.328us	void mshadow::cuda::SoftmaxKernel<int=8,

Figure 2: Optimization 2: Profiler Output

Compared with the first optimization, we can tell that the performance of the second layer is improved while the performance of the first layer is even worse. This is because loading global memory is unordered in the kernel, which outperforms the benefits of the efficiency of the matrix multiplication. This leads to our further improvement of using two different kernels for two layers in the next part.

4 Optimization 3: Multiple kernel implementations for different layer sizes

4.1 Description

In the optimization 2, we can find that the second layer is accelerated from 0.115083 to 0.090749 but the speed of the first layer is slowing down from 0.026521 to 0.044929. So that we can implement two different kernel for different input channel size. The reason why the first optimization is better than optimization 2 for layer 1 is that the input channel size of layer 1 is 1 which means that we can reduce a loop for optimization 1. On the other hand, the structure of layer 2 is much more complex which means that unrolling it into matrix and applying tiled-matrix multiplication can reduce the loop and speed up the calculation.

4.2 Profiler Output

```
-----
New Inference
Op Time: 0.026521
Op Time: 0.090749
Correctness: 0.7955 Model: eecs498
==2397852== Profiling application: python2 submit/submission.py
==2397852== Profiling result:
   Type  Time(%)   Time     Calls   Avg       Min       Max  Name
GPU activities:  42.03%  103.99ms      8 12.999ms  1.1520us 101.89ms [CUDA memcpy HtoD]
                36.67%  90.730ms     1 90.730ms  90.730ms 90.730ms mxnet::op::forward_kernel2(float*, floa
                10.70%  26.472ms     1 26.472ms  26.472ms 26.472ms mxnet::op::forward_kernel1(float*, floa
                4.50%  11.132ms     2 5.5658ms  1.8929ms 9.2388ms void mshadow::cuda::MapPlanLargeKernel
                2.40%  5.9461ms     1 5.9461ms  5.9461ms 5.9461ms volta_sgemm_32x32_sliced1x4_tn
                2.16%  5.3342ms     2 2.6671ms  18.784us 5.3154ms void op_generic_tensor_kernel<int=1, f
                1.33%  3.2961ms     1 3.2961ms  3.2961ms 3.2961ms void cudnn::ops::pooling_fw_4d_kernel<
                0.12%  294.08us     1 294.08us  294.08us 294.08us void mshadow::cuda::MapPlanLargeKernel
```

Figure 3: Optimization 3: Profiler Output

In this optimization, we in total use two kernel functions and we can see that most of the time is used for memory access which shows that the main direction for further optimization lies in improving shared memory utility and one possible solution is sweeping through different TILE_WIDTH for both kernel functions.

We can see that in this stage, the two different layers all find the right algorithm for them and reached the current optimal solution and the next stage is sweeping through various parameters.

5 Optimization 4: Sweeping various parameters

5.1 Description

The following two tables show the results for different TILE_WIDTH for the two kernel functions. For the first kernel function, we use different tile width and height and for the second one, we use square tile because of the limitation of the algorithm.

Finally, we can find that the optimal parameters are

$$\text{TILE_WIDTH1_X} = 34$$

$$\text{TILE_WIDTH1_Y} = 14$$

$$\text{TILE_WIDTH2} = 24$$

TILE_WIDTH1_X	TILE_WIDTH1_Y	time
32	11	0.32424
32	12	0.32734
32	14	0.32994
32	16	0.34240
33	11	0.23792
33	12	0.24426
33	14	0.23819
33	16	0.25780
34	11	0.23770
34	12	0.24354
34	14	0.23716
34	16	0.25684

Table 1: Kernel1 sweep

TILE_WIDTH2	time
16	0.99450
20	1.29632
24	0.69244
28	0.92760
32	0.90292

Table 2: Kernel2 sweep

5.2 Profiler Output

```

New Inference
Op Time: 0.023713
Op Time: 0.073145
Correctness: 0.7955 Model: eeecs498
==861058== Profiling application: python2 submit/submission.py
==861058== Profiling result:
   Type  Time(%)   Time     Calls   Avg       Min       Max  Name
GPU activities:  40.90%  85.516ms      8  10.689ms  1.0880us  83.421ms  [CUDA memcpy HtoD]
                34.97%  73.127ms      1  73.127ms  73.127ms  73.127ms  mxnet::op::forward_kernel2(float
                11.31%  23.658ms      1  23.658ms  23.658ms  23.658ms  mxnet::op::forward_kernel1(float
                5.33%  11.148ms      2  5.5742ms  1.9115ms  9.2370ms  void mshadow::cuda::MapPlanLarge
                3.09%   6.4531ms      1   6.4531ms  6.4531ms  6.4531ms  volta_sgemm_32x32_sliced1x4_tn
                2.56%   5.3547ms      2   2.6773ms  19.264us  5.3354ms  void op_generic_tensor_kernel<in

```

Figure 4: Optimization 4: Profiler Output

In this optimization, we in total use two kernel functions and we can see that most of the time is used for memory access.

A Code of Optimization 1

```
1 #ifndef MXNET_OPERATOR_NEW_FORWARD_CUH
2 #define MXNET_OPERATOR_NEW_FORWARD_CUH
3 #define TILE_WIDTH 28
4
5 #include <mxnet/base.h>
6
7 namespace mxnet
8 {
9     namespace op
10    {
11
12    __global__ void forward_kernel(float *y, const float *x, const float
        *k, const int B, const int M, const int C, const int H, const
        int W, const int K)
13    {
14        /*
15         Modify this function to implement the forward pass described in
            Chapter 16.
16         We have added an additional dimension to the tensors to support
            an entire mini-batch
17         The goal here is to be correct AND fast.
18         We have some nice #defs for you below to simplify indexing. Feel
            free to use them, or create your own.
19         */
20
21        const int H_out = H - K + 1;
22        const int W_out = W - K + 1;
23        // (void)H_out; // silence declared but never referenced warning
        . remove this line when you start working
24        // (void)W_out; // silence declared but never referenced warning
        . remove this line when you start working
25
26        // __shared__ float x_tile[32][28][28];
27
28        // int dx = blockDim.x; int dy = blockDim.y; int dz = blockDim.z
        ;
29
30        // int map = bx * C * 28 * 28 + ty * 28 + tx;
31
32        // for (int c = 0; c < C; c++)
33        // {
34        //     x_tile[c][ty][tx] = x[map + c * 28 * 28];
35        // }
36        // __syncthreads();
37
38        // An example use of these macros:
39        // float a = y4d(0,0,0,0)
40        // y4d(0,0,0,0) = a
```

```

41 // N -> b
42 #define y4d(i3, i2, i1, i0) y[(i3) * (M * H_out * W_out) + (i2) * (
    H_out * W_out) + (i1) * (W_out) + i0]
43 #define x4d(i3, i2, i1, i0) x[(i3) * (C * H * W) + (i2) * (H * W) +
    (i1) * (W) + i0]
44 #define k4d(i3, i2, i1, i0) k[(i3) * (C * K * K) + (i2) * (K * K) +
    (i1) * (K) + i0]
45
46 int n, m, h, w, c, p, q;
47 int W_grid = ceil(W_out/float(TILE_WIDTH));
48 n = blockIdx.x;
49 m = blockIdx.y;
50 h = (blockIdx.z / W_grid) * TILE_WIDTH + threadIdx.y;
51 w = (blockIdx.z % W_grid) * TILE_WIDTH + threadIdx.x;
52 float acc = 0;
53 if ((w < W_out) && (h < H_out))
54 {
55     // #pragma unroll(12)
56     for (c = 0; c < C; c++)
57     { // sum over all input channels
58         // #pragma unroll(7)
59         for (p = 0; p < K; p++) // loop over KxK filter
60             // #pragma unroll(7)
61             for (q = 0; q < K; q++)
62                 acc = acc + x4d(n, c, h + p, w + q) * k4d(m, c,
                    p, q);
63     }
64     y4d(n, m, h, w) = acc;
65
66 #undef y4d
67 #undef x4d
68 #undef k4d
69 }
70
71 /*
72     This function is called by new-inl.h
73     Any code you write should be executed by this function.
74     For ECE408, we only expect the float version of the operator to
75     be called, so here we specialize with only floats.
76 */
77 template <
78 void forward<gpu, float>(mshadow::Tensor<gpu, 4, float> &y, const
    mshadow::Tensor<gpu, 4, float> &x, const mshadow::Tensor<gpu, 4,
    float> &w)
79 {
80     // // Use mxnet's CHECK_EQ to do assertions.
81     // // Remove this assertion when you do your implementation!
82     // CHECK_EQ(0, 1) << "Remove this line and replace with your
    implementation";

```

```

83
84 // output M*W*H
85 // #images in batch
86 const int B = x.shape_[0];
87 // #output hidden layer size #channels
88 const int M = y.shape_[1]; // num_filter
89 // input hidden layer size
90 const int C = x.shape_[1];
91 // height
92 const int H = x.shape_[2];
93 // width
94 const int W = x.shape_[3];
95 // kernel size
96 const int K = w.shape_[3];
97
98 const int H_out = H - K + 1;
99 const int W_out = W - K + 1;
100 int W_grid = ceil(W_out/(float(TILE_WIDTH)));
101 int H_grid = ceil(H_out/(float(TILE_WIDTH)));
102 int Z = H_grid * W_grid;
103
104 // dim3 gridDim((B + 511) / 512);
105 // dim3 blockDim(512);
106 dim3 blockDim(TILE_WIDTH, TILE_WIDTH, 1);
107 dim3 gridDim(B, M, Z);
108
109 // cudaMemcpyToSymbol(kernel, w.dptr_, sizeof(float) * M * C * K
110 //    * K);
111
112 MSHADOW_CUDA_CALL(cudaDeviceSynchronize());
113 forward_kernel<<<gridDim, blockDim>>>(y.dptr_, x.dptr_, w.dptr_,
114     B, M, C, H, W, K);
115 MSHADOW_CUDA_CALL(cudaDeviceSynchronize());
116 }
117
118 /*
119 This tells mxnet how to do an op when it's not a float.
120 This is not used in the ECE408 project
121 */
122 template <typename gpu, typename DType>
123 void forward(mshadow::Tensor<gpu, 4, DType> &y, const mshadow::
124     Tensor<gpu, 4, DType> &x, const mshadow::Tensor<gpu, 4, DType> &w
125     )
126 {
127     assert(0 && "No forward implementation for other datatypes
128         needed for ECE408");
129 }
130 }
131 }
132 }
133 }

```

```
128 #endif
```

B Code of Optimization 2

```
1
2 #ifndef MXNET_OPERATOR_NEW_FORWARD_CUH
3 #define MXNET_OPERATOR_NEW_FORWARD_CUH
4 #define TILE_WIDTH1 24
5 #define TILE_WIDTH2 32
6
7 #include <mxnet/base.h>
8
9 namespace mxnet
10 {
11     namespace op
12     {
13
14         __global__ void forward_kernel1(float *y, const float *x, const
            float *k, const int B, const int M, const int C, const int H,
            const int W, const int K)
15         {
16             /*
17              Modify this function to implement the forward pass described in
18              Chapter 16.
19              We have added an additional dimension to the tensors to support
20              an entire mini-batch
21              The goal here is to be correct AND fast.
22              We have some nice #defs for you below to simplify indexing. Feel
23              free to use them, or create your own.
24              */
25
26             const int H_out = H - K + 1;
27             const int W_out = W - K + 1;
28
29             // An example use of these macros:
30             // float a = y4d(0,0,0,0)
31             // y4d(0,0,0,0) = a
32             #define y4d(i3, i2, i1, i0) y[(i3) * (M * H_out * W_out) + (i2) * (
                H_out * W_out) + (i1) * (W_out) + i0]
33             #define x4d(i3, i2, i1, i0) x[(i3) * (C * H * W) + (i2) * (H * W) +
                (i1) * (W) + i0]
34             #define k4d(i3, i2, i1, i0) k[(i3) * (C * K * K) + (i2) * (K * K) +
                (i1) * (K) + i0]
35
36             __shared__ float subTileA[TILE_WIDTH1][TILE_WIDTH1];
37             __shared__ float subTileB[TILE_WIDTH1][TILE_WIDTH1];
38
39             int bx = blockIdx.x; int by = blockIdx.y;
40             int tx = threadIdx.x; int ty = threadIdx.y;
```

```

38
39     int bz = blockIdx.z;
40     // int dx = blockDim.x; int dy = blockDim.y;
41
42     int Row = by*TILE_WIDTH1 + ty;
43     int Col = bx*TILE_WIDTH1 + tx;
44
45     int numAColumns = C * K * K;
46     int numBColumns = H_out * W_out;
47
48     float res = 0;
49     int it = ceil(1.0 * numAColumns / TILE_WIDTH1);
50
51     for (int i = 0; i < it; i++)
52     {
53         int tile_x = i*TILE_WIDTH1 + tx;
54         int tile_y = i*TILE_WIDTH1 + ty;
55         int k_idx = tile_x % (K*K);
56         int x_idx = tile_y % (K*K);
57
58         if ( tile_x < numAColumns && Row < M)
59             subTileA[ty][tx] = k4d(Row, tile_x/(K*K), k_idx / K,
60                                     k_idx % K);
61         else
62             subTileA[ty][tx] = 0.0;
63
64         if ( tile_y < numAColumns && Col < numBColumns)
65             subTileB[ty][tx] = x4d(bz, tile_y/(K*K), Col/W_out +
66                                     x_idx/K, Col % W_out + x_idx % K);
67         else
68             subTileB[ty][tx] = 0.0;
69
70         __syncthreads();
71
72         for (int i = 0; i < TILE_WIDTH1; i++)
73             res += subTileA[ty][i] * subTileB[i][tx];
74
75         __syncthreads();
76     }
77
78     if (Row < M && Col < numBColumns)
79         y4d(bz, Row, Col / W_out, Col % W_out) = res;
80
81 #undef y4d
82 #undef x4d
83 #undef k4d
84 }
85
86 --global-- void forward_kernel2(float *y, const float *x, const
87                                float *k, const int B, const int M, const int C, const int H,

```

```

85  const int W, const int K)
86  {
87      /*
88       * Modify this function to implement the forward pass described in
89       * Chapter 16.
90       * We have added an additional dimension to the tensors to support
91       * an entire mini-batch
92       * The goal here is to be correct AND fast.
93       * We have some nice #defs for you below to simplify indexing. Feel
94       * free to use them, or create your own.
95       */
96      const int H_out = H - K + 1;
97      const int W_out = W - K + 1;
98
99      // An example use of these macros:
100     // float a = y4d(0,0,0,0)
101     // y4d(0,0,0,0) = a
102     #define y4d(i3, i2, i1, i0) y[(i3) * (M * H_out * W_out) + (i2) * (
103         H_out * W_out) + (i1) * (W_out) + i0]
104     #define x4d(i3, i2, i1, i0) x[(i3) * (C * H * W) + (i2) * (H * W) +
105         (i1) * (W) + i0]
106     #define k4d(i3, i2, i1, i0) k[(i3) * (C * K * K) + (i2) * (K * K) +
107         (i1) * (K) + i0]
108
109     __shared__ float subTileA[TILE_WIDTH2][TILE_WIDTH2];
110     __shared__ float subTileB[TILE_WIDTH2][TILE_WIDTH2];
111
112     int bx = blockIdx.x; int by = blockIdx.y;
113     int tx = threadIdx.x; int ty = threadIdx.y;
114
115     int bz = blockIdx.z;
116     // int dx = blockDim.x; int dy = blockDim.y;
117
118     int Row = by*TILE_WIDTH2 + ty;
119     int Col = bx*TILE_WIDTH2 + tx;
120
121     int numAColumns = C * K * K;
122     int numBColumns = H_out * W_out;
123
124     float res = 0;
125     int it = ceil(1.0 * numAColumns / TILE_WIDTH2);
126
127     for (int i = 0; i < it; i++)
128     {
129         int tile_x = i*TILE_WIDTH2 + tx;
130         int tile_y = i*TILE_WIDTH2 + ty;
131         int k_idx = tile_x % (K*K);
132         int x_idx = tile_y % (K*K);

```

```

128         if ( tile_x < numAColumns && Row < M)
129             subTileA[ty][tx] = k4d(Row, tile_x/(K*K), k_idx / K,
130                                     k_idx % K);
131         else
132             subTileA[ty][tx] = 0.0;
133
134         if ( tile_y < numAColumns && Col < numBColumns)
135             subTileB[ty][tx] = x4d(bz, tile_y/(K*K), Col/W_out +
136                                     x_idx/K, Col % W_out + x_idx % K);
137         else
138             subTileB[ty][tx] = 0.0;
139
140         __syncthreads();
141
142         for (int i = 0; i < TILE_WIDTH2; i++)
143             res += subTileA[ty][i] * subTileB[i][tx];
144
145         __syncthreads();
146     }
147
148     if (Row < M && Col < numBColumns)
149         y4d(bz, Row, Col / W_out, Col % W_out) = res;
150
151 #undef y4d
152 #undef x4d
153 #undef k4d
154 }
155
156 /*
157     This function is called by new-inl.h
158     Any code you write should be executed by this function.
159     For ECE408, we only expect the float version of the operator to
160     be called, so here we specialize with only floats.
161 */
162 template <>
163 void forward<gpu, float>(mshadow::Tensor<gpu, 4, float> &y, const
164                         mshadow::Tensor<gpu, 4, float> &x, const mshadow::Tensor<gpu, 4,
165                         float> &w)
166 {
167     // // Use mxnet's CHECK_EQ to do assertions.
168     // // Remove this assertion when you do your implementation!
169     // CHECK_EQ(0, 1) << "Remove this line and replace with your
170     // implementation";
171
172     // output M*W*H
173     // #images in batch
174     const int B = x.shape_[0];
175     // #output hidden layer size #channels
176     const int M = y.shape_[1]; // num_filter

```

```

172 // input hidden layer size
173 const int C = x.shape_[1];
174 // height
175 const int H = x.shape_[2];
176 // width
177 const int W = x.shape_[3];
178 // kernel size
179 const int K = w.shape_[3];
180
181 const int H_out = H - K + 1;
182 const int W_out = W - K + 1;
183
184 if(M == 24)
185 {
186     int gx = ceil(1.0 * H_out * W_out / TILE_WIDTH2);
187     int gy = ceil(1.0 * M / TILE_WIDTH2);
188
189     dim3 blockDim(TILE_WIDTH2, TILE_WIDTH2, 1);
190     dim3 gridDim(gx, gy, B);
191
192     MSHADOW_CUDA_CALL(cudaDeviceSynchronize());
193     forward_kernel2<<<gridDim, blockDim>>>(y.dptr_, x.dptr_, w.
194         dptr_, B, M, C, H, W, K);
195     MSHADOW_CUDA_CALL(cudaDeviceSynchronize());
196 }
197 else
198 {
199     int gx = ceil(1.0 * H_out * W_out / TILE_WIDTH1);
200     int gy = ceil(1.0 * M / TILE_WIDTH1);
201
202     dim3 blockDim(TILE_WIDTH1, TILE_WIDTH1, 1);
203     dim3 gridDim(gx, gy, B);
204
205     MSHADOW_CUDA_CALL(cudaDeviceSynchronize());
206     forward_kernel1<<<gridDim, blockDim>>>(y.dptr_, x.dptr_, w.
207         dptr_, B, M, C, H, W, K);
208     MSHADOW_CUDA_CALL(cudaDeviceSynchronize());
209 }
210 }
211
212 /*
213     This tells mxnet how to do an op when it's not a float.
214     This is not used in the ECE408 project
215 */
216 template <typename gpu, typename DType>
217 void forward(mshadow::Tensor<gpu, 4, DType> &y, const mshadow::
218     Tensor<gpu, 4, DType> &x, const mshadow::Tensor<gpu, 4, DType> &w
219 )
220 {
221     assert(0 && "No forward implementation for other datatypes

```

```

218     }
219     }
220     }
221
222     needed for ECE408");
#endif

```

C Code of Optimization 3

```

1
2 #ifndef MXNET_OPERATOR_NEW_FORWARD_CUH
3 #define MXNET_OPERATOR_NEW_FORWARD_CUH
4 #define TILE_WIDTH1_X 24
5 #define TILE_WIDTH1_Y 24
6 #define TILE_WIDTH2 28
7
8 #include <mxnet/base.h>
9
10
11 namespace mxnet
12 {
13     namespace op
14     {
15
16         __constant__ float kernel[588];
17         __global__ void forward_kernel1(float *y, const float *x, const int
            M, const int H, const int W, const int K, const int W_grid, const
            int H_out, const int W_out)
18     {
19         int p, q;
20         int n = blockIdx.x;
21         int m = blockIdx.y;
22         int bz = blockIdx.z;
23         int h = (bz / W_grid) * TILE_WIDTH1_Y + threadIdx.y;
24         int w = (bz % W_grid) * TILE_WIDTH1_X + threadIdx.x;
25         float acc = 0;
26         if ((w < W_out) && (h < H_out))
27         {
28             #pragma unroll
29             for (p = 0; p < K; p++)
30                 #pragma unroll
31                 for (q = 0; q < K; q++)
32                     acc = acc + x[n * H * W + (h + p) * W + w + q] *
                        kernel[m * K * K + p * K + q];
33             y[n * M * H_out * W_out + m * H_out * W_out + h * W_out + w]
                = acc;
34         }
35     }
36

```

```

37 __global__ void forward_kernel2(float *y, const float *x, const
    float *k, const int M, const int C, const int H, const int W,
    const int K)
38 {
39     const int H_out = H - K + 1;
40     const int W_out = W - K + 1;
41     int KK = K*K;
42
43 #define y4d(i3, i2, i1, i0) y[(i3) * (M * H_out * W_out) + (i2) * (
    H_out * W_out) + (i1) * (W_out) + i0]
44 #define x4d(i3, i2, i1, i0) x[(i3) * (C * H * W) + (i2) * (H * W) +
    (i1) * (W) + i0]
45 #define k4d(i3, i2, i1, i0) k[(i3) * (C * KK) + (i2) * (KK) + (i1) *
    (K) + i0]
46
47 __shared__ float subTileA[TILE_WIDTH2][TILE_WIDTH2];
48 __shared__ float subTileB[TILE_WIDTH2][TILE_WIDTH2];
49
50 int tx = threadIdx.x; int ty = threadIdx.y;
51 int bz = blockIdx.z;
52
53 int Col = blockIdx.x*TILE_WIDTH2 + tx;
54 int Row = blockIdx.y*TILE_WIDTH2 + ty;
55
56 int numAColumns = C * KK;
57 float res = 0;
58 int cw1 = Col/W_out;
59 int cw2 = Col%W_out;
60 int tile_x, tile_y, k_idx, x_idx;
61
62 for (int i = 0; i < numAColumns; i+=TILE_WIDTH2)
63 {
64     tile_x = i + tx;
65     tile_y = i + ty;
66     k_idx = tile_x % (KK);
67     x_idx = tile_y % (KK);
68
69     subTileA[ty][tx] = k4d(ty, tile_x/(KK), k_idx / K, k_idx % K
    );
70
71     if (tile_y < numAColumns)
72         subTileB[ty][tx] = x4d(bz, tile_y/(KK), cw1 + x_idx/K,
    cw2 + x_idx % K);
73     else
74         subTileB[ty][tx] = 0.0;
75
76     __syncthreads();
77
78 #pragma unroll
79     for (int j = 0; j < TILE_WIDTH2; j++)

```

```

80         res += subTileA[ty][j] * subTileB[j][tx];
81
82     __syncthreads();
83 }
84
85 if ( Row < M && Col < H_out * W_out)
86     y4d(bz, ty, cw1, cw2) = res;
87
88 #undef y4d
89 #undef x4d
90 #undef k4d
91 }
92
93 /*
94  This function is called by new-inl.h
95  Any code you write should be executed by this function.
96  For ECE408, we only expect the float version of the operator to
97  be called, so here we specialize with only floats.
98 */
99 template<>
100 void forward<gpu, float>(mshadow::Tensor<gpu, 4, float> &y, const
    mshadow::Tensor<gpu, 4, float> &x, const mshadow::Tensor<gpu, 4,
    float> &w)
101 {
102     // output M*W*H
103     // #images in batch
104     const int B = x.shape_[0];
105     // #output hidden layer size #channels
106     const int M = y.shape_[1]; // num_filter
107     // input hidden layer size
108     const int C = x.shape_[1];
109     // height
110     const int H = x.shape_[2];
111     // width
112     const int W = x.shape_[3];
113     // kernel size
114     const int K = w.shape_[3];
115
116     const int H_out = H - K + 1;
117     const int W_out = W - K + 1;
118
119     if (C == 1)
120     {
121         int W_grid = ceil(1.0 * W_out / TILE_WIDTH1_X);
122         int H_grid = ceil(1.0 * H_out / TILE_WIDTH1_Y);
123
124         dim3 blockDim(TILE_WIDTH1_X, TILE_WIDTH1_Y);
125         dim3 gridDim(B, M, H_grid * W_grid);
126
127         cudaMemcpyToSymbol(kernel, w.dptr_, sizeof(float) * C * M *

```

```

127         K * K);
128         forward_kernel1<<<gridDim, blockDim>>>(y.dptr_, x.dptr_, M,
129         H, W, K, W_grid, H_out, W_out);
130         MSHADOW_CUDA_CALL(cudaDeviceSynchronize());
131     }
132     else
133     {
134         int gx = ceil(1.0 * H_out * W_out / TILE_WIDTH2);
135         int gy = ceil(1.0 * M / TILE_WIDTH2);
136
137         dim3 blockDim(TILE_WIDTH2, TILE_WIDTH2);
138         dim3 gridDim(gx, gy, B);
139
140         forward_kernel2<<<gridDim, blockDim>>>(y.dptr_, x.dptr_, w.
141         dptr_, M, C, H, W, K);
142         MSHADOW_CUDA_CALL(cudaDeviceSynchronize());
143     }
144 }
145
146 /*
147  This tells mxnet how to do an op when it's not a float.
148  This is not used in the ECE408 project
149 */
150 template <typename gpu, typename DType>
151 void forward(mshadow::Tensor<gpu, 4, DType> &y, const mshadow::
152     Tensor<gpu, 4, DType> &x, const mshadow::Tensor<gpu, 4, DType> &w
153     )
154 {
155     assert(0 && "No forward implementation for other datatypes
156         needed for ECE408");
157 }
158 }
159 }
160 #endif

```

D Code of Optimization 4

```

1
2 #ifndef MXNET_OPERATOR_NEW_FORWARD_CUH
3 #define MXNET_OPERATOR_NEW_FORWARD_CUH
4 #define TILE_WIDTH1_X 34
5 #define TILE_WIDTH1_Y 14
6 #define TILE_WIDTH2 24
7
8 #include <mxnet/base.h>
9
10

```

```

11 namespace mxnet
12 {
13 namespace op
14 {
15
16 __constant__ float kernel[588];
17 __global__ void forward_kernel1(float *y, const float *x, const int
    M, const int H, const int W, const int K, const int W_grid, const
    int H_out, const int W_out)
18 {
19     int p, q;
20     int n = blockIdx.x;
21     int m = blockIdx.y;
22     int bz = blockIdx.z;
23     int h = (bz / W_grid) * TILE_WIDTH1_Y + threadIdx.y;
24     int w = (bz % W_grid) * TILE_WIDTH1_X + threadIdx.x;
25     float acc = 0;
26     if ((w < W_out) && (h < H_out))
27     {
28         #pragma unroll
29         for (p = 0; p < K; p++)
30             #pragma unroll
31             for (q = 0; q < K; q++)
32                 acc = acc + x[n * H * W + (h + p) * W + w + q] *
33                     kernel[m * K * K + p * K + q];
34         y[n * M * H_out * W_out + m * H_out * W_out + h * W_out + w]
35         = acc;
36     }
37 }
38
39 __global__ void forward_kernel2(float *y, const float *x, const
    float *k, const int M, const int C, const int H, const int W,
    const int K)
40 {
41     const int H_out = H - K + 1;
42     const int W_out = W - K + 1;
43     int KK = K*K;
44
45 #define y4d(i3, i2, i1, i0) y[(i3) * (M * H_out * W_out) + (i2) * (
    H_out * W_out) + (i1) * (W_out) + i0]
46 #define x4d(i3, i2, i1, i0) x[(i3) * (C * H * W) + (i2) * (H * W) +
    (i1) * (W) + i0]
47 #define k4d(i3, i2, i1, i0) k[(i3) * (C * KK) + (i2) * (KK) + (i1) *
    (K) + i0]
48
49 __shared__ float subTileA[TILE_WIDTH2][TILE_WIDTH2];
50 __shared__ float subTileB[TILE_WIDTH2][TILE_WIDTH2];
51
52     int tx = threadIdx.x; int ty = threadIdx.y;
53     int bz = blockIdx.z;

```

```

52
53     int Col = blockIdx.x*TILE_WIDTH2 + tx;
54     int Row = blockIdx.y*TILE_WIDTH2 + ty;
55
56     int numAColumns = C * KK;
57     float res = 0;
58     int cw1 = Col/W_out;
59     int cw2 = Col%W_out;
60     int tile_x , tile_y , k_idx , x_idx;
61
62     for (int i = 0; i < numAColumns; i+=TILE_WIDTH2)
63     {
64         tile_x = i + tx;
65         tile_y = i + ty;
66         k_idx = tile_x % (KK);
67         x_idx = tile_y % (KK);
68
69         subTileA[ty][tx] = k4d(ty, tile_x/(KK), k_idx / K, k_idx % K
70                                );
71
72         if ( tile_y < numAColumns)
73             subTileB[ty][tx] = x4d(bz, tile_y/(KK), cw1 + x_idx/K,
74                                     cw2 + x_idx % K);
75         else
76             subTileB[ty][tx] = 0.0;
77
78         __syncthreads();
79
80         #pragma unroll
81         for (int j = 0; j < TILE_WIDTH2; j++)
82             res += subTileA[ty][j] * subTileB[j][tx];
83
84         __syncthreads();
85     }
86
87     if ( Row < M && Col < H_out * W_out)
88         y4d(bz, ty, cw1, cw2) = res;
89
90 #undef y4d
91 #undef x4d
92 #undef k4d
93 }
94
95 /*
96     This function is called by new-inl.h
97     Any code you write should be executed by this function.
98     For ECE408, we only expect the float version of the operator to
99     be called , so here we specialize with only floats.
100 */
101 template <

```

```

99 void forward<gpu, float>(mshadow::Tensor<gpu, 4, float> &y, const
    mshadow::Tensor<gpu, 4, float> &x, const mshadow::Tensor<gpu, 4,
    float> &w)
100 {
101     // output M*W*H
102     // #images in batch
103     const int B = x.shape_[0];
104     // #output hidden layer size #channels
105     const int M = y.shape_[1]; // num_filter
106     // input hidden layer size
107     const int C = x.shape_[1];
108     // height
109     const int H = x.shape_[2];
110     // width
111     const int W = x.shape_[3];
112     // kernel size
113     const int K = w.shape_[3];
114
115     const int H_out = H - K + 1;
116     const int W_out = W - K + 1;
117
118     if(C == 1)
119     {
120         int W_grid = ceil(1.0 * W_out/TILE_WIDTH1_X);
121         int H_grid = ceil(1.0 * H_out/TILE_WIDTH1_Y);
122
123         dim3 blockDim(TILE_WIDTH1_X, TILE_WIDTH1_Y);
124         dim3 gridDim(B, M, H_grid * W_grid);
125
126         cudaMemcpyToSymbol(kernel, w.dptr_, sizeof(float) * C * M *
            K * K);
127
128         forward_kernel1<<<gridDim, blockDim>>>(y.dptr_, x.dptr_, M,
            H, W, K, W_grid, H_out, W_out);
129         MSHADOW_CUDA_CALL(cudaDeviceSynchronize());
130     }
131     else
132     {
133         int gx = ceil(1.0 * H_out * W_out / TILE_WIDTH2);
134         int gy = ceil(1.0 * M / TILE_WIDTH2);
135
136         dim3 blockDim(TILE_WIDTH2, TILE_WIDTH2);
137         dim3 gridDim(gx, gy, B);
138
139         forward_kernel2<<<gridDim, blockDim>>>(y.dptr_, x.dptr_, w.
            dptr_, M, C, H, W, K);
140         MSHADOW_CUDA_CALL(cudaDeviceSynchronize());
141     }
142 }
143

```

```

144  /*
145     This tells mxnet how to do an op when it's not a float.
146     This is not used in the ECE408 project
147  */
148  template <typename gpu, typename DType>
149  void forward(mshadow::Tensor<gpu, 4, DType> &y, const mshadow::
    Tensor<gpu, 4, DType> &x, const mshadow::Tensor<gpu, 4, DType> &w
    )
150  {
151     assert(0 && "No forward implementation for other datatypes
        needed for ECE408");
152 }
153 }
154 }
155
156 #endif

```